



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

INTEGRATED HOSPITAL MANAGEMENT SYSTEM WITH PATIENT RECORDS, DOCTOR SCHEDULING AND BILLING

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements

for the Course of

CSA0208 – C Programming

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ELECTRONICS AND COMMUNICATION ENGINEERING

Submitted by

Thilagavathi. K (192312301)

Harisha S K (192312311)

Usha Rani C (192312304)

Under the Supervision of

Dr. V. Jayasudha

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

SEPTEMBER 2026

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "Integrated Hospital Management System with Patient Records, Doctor Scheduling and Billing" has been carried out by Thilagavathi. K (192312301), Harisha S K (192312311), Usha Rani C (192312304) under the supervision of Dr. V. Jayasudha and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech Electronics and Communication Engineering programme (Course: CSA0208 – C Programming) at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. M. Shakila,
Professor
Department of CSE
Saveetha School of Engineering
SIMATS

COURSE FACULTY

Dr. V. Jayasudha
Professor
Department of CSE
Saveetha School of Engineering
SIMATS.

Submitted for the project work Viva-Voce held on: _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

We, Thilagavathi. K, Harisha S K, Usha Rani C, of the Department of Electronics and Communication Engineering, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "Integrated Hospital Management System with Patient Records, Doctor Scheduling and Billing" is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date:

Signature of the Students with Names

Individual Contribution Statement

Student Name / Reg. No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
Thilagavathi. K (192312301)	C code design for Patient Records module; menu-driven CLI logic	Struct design, add/view/search/update/delete patient functions	Unit testing of patient module; validation of file read/write	Chapters 1, 3 (3.1–3.4)	34%
Harisha S K (192312311)	C code design for Doctor Scheduling and Appointment module	Doctor struct, schedule/appointment conflict-checking logic	Testing of scheduling module; conflict scenarios	Chapters 3 (3.5–3.7), 4 (4.1–4.4)	33%
Usha Rani C (192312304)	C code design for Billing module and file-based data persistence	Bill struct, invoice calculation, integration of all modules	System-level integration testing; results compilation	Chapters 4 (4.5–4.8), 5, References	33%

Total: 100%

ABSTRACT

Hospitals and small clinics that continue to rely on manual, paper-based methods for maintaining patient records, doctor schedules, and billing frequently encounter data duplication, delayed information retrieval, appointment conflicts, and billing errors. These inefficiencies directly affect the quality and speed of patient care and increase the administrative burden on hospital staff. There is, therefore, a clear engineering need for a lightweight, dependable digital system that can consolidate these three core hospital functions on modest computing resources, particularly in small-scale healthcare setups that cannot immediately afford large, database-driven Hospital Information Systems.

This project proposes and implements an Integrated Hospital Management System (IHMS) developed entirely in the C programming language as a menu-driven, file-based console application. The system is organised into three interacting modules — Patient Records Management, Doctor Scheduling and Appointment Management, and Billing Management — built around dedicated C structures (struct Patient, struct Doctor, and struct Bill) and persisted using sequential binary/text files, removing the need for an external database engine. The methodology followed a structured, modular programming approach: requirements were gathered from typical hospital front-office workflows, the system architecture and data structures were designed and evaluated against alternative storage strategies, the modules were implemented incrementally in GCC-compiled C, and each module was verified through functional black-box test cases covering normal, boundary, and invalid-input scenarios.

The implemented prototype successfully allows hospital staff to add, view, search, update, and delete patient records; register doctors and schedule appointments while automatically detecting double-booking conflicts; and generate itemised bills that sum consultation, medicine, and other service charges into a final payable amount stored against each patient. Testing confirmed that all core functional requirements were met, with correct outputs obtained across the designed test cases and acceptable response times for the target record volumes. The project demonstrates that fundamental C-language constructs — structures, arrays, file handling, and modular functions — are sufficient to build a functionally complete, centralised hospital management prototype that reduces paperwork, minimises manual calculation errors, and improves the accessibility of patient and administrative information, while also identifying clear directions for future enhancement through database migration, AI-assisted analytics, and mobile/online integration.

Keywords: Hospital Management System, C Programming, File Handling, Patient Records, Doctor Scheduling, Billing Automation

TABLE OF CONTENTS

Sl. No.	Title	Page No.
i	CERTIFICATE FROM THE SUPERVISOR	ii
ii	DECLARATION BY THE CANDIDATE	iii
iii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vii
vi	LIST OF TABLES	viii
vii	LIST OF ABBREVIATIONS	ix
1	CHAPTER 1: INTRODUCTION AND ENGINEERING PROBLEM	1
2	CHAPTER 2: LITERATURE REVIEW AND EXISTING SOLUTIONS	3
3	CHAPTER 3: ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY	6
4	CHAPTER 4: IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT	16
5	CHAPTER 5: CONCLUSION, FUTURE WORK AND REFLECTION	18
	REFERENCES	19
	APPENDICES	21

LIST OF FIGURES

Figure No.	Figure Name	Page No.
3.1	System Architecture of the Integrated Hospital Management System	7
3.2	Relationship Between Core Data Structures (Patient, Doctor, Bill)	8
4.1	Adding Patient	20
4.2	Adding Doctor	20
4.3	Generating Bill	21
4.4	Whole Dashboard	21

LIST OF TABLES

Table No.	Table Name	Page No.
2.1	Comparative Analysis of Existing Approaches	3
3.1	Stakeholder / User Requirements	5
3.2	Functional and Non-Functional Requirements	5
3.3	Constraints and Applicable Standards	6
3.4	Design Specifications	6
3.5	Alternative Solutions Evaluation Matrix	6
3.6	Trade-off Analysis	8
3.7	Sustainability, Ethics, Safety and Security Considerations	9
3.8	Risk Register	9
4.1	Tools / Software Used	11
4.2	Test Plan and Verification	11
4.3	Test Results Summary	12
4.4	Comparison with Existing Solutions & Achievement of Objectives	13
4.5	Project Milestone / Timeline	14
4.6	Roles and Responsibilities	14
4.7	Estimated Project Cost / Effort	14

LIST OF ABBREVIATIONS / SYMBOLS

Symbol / Abbreviation	Description
HMS	Hospital Management System
IHMS	Integrated Hospital Management System
C	C Programming Language (ISO/IEC 9899)
CLI	Command-Line Interface
GCC	GNU Compiler Collection
IDE	Integrated Development Environment
EMR	Electronic Medical Record
ID	Identification Number
I/O	Input / Output
OS	Operating System
.dat	Data file extension used for persistent file storage

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Hospitals and clinics handle three closely related streams of information on a daily basis: patient medical and personal details, doctor availability and appointment schedules, and financial billing records for services rendered. Traditionally, particularly in small and medium-sized healthcare establishments, these records have been maintained using paper registers or, at best, isolated spreadsheet files. Each department — the reception desk, the consulting doctors, and the billing counter — tends to keep its own version of the truth, with little or no real-time synchronisation between them. The C programming language, with its structures, arrays, and low-level file-handling capabilities, provides a lightweight and resource-efficient way of building a first working digital system to consolidate these functions without requiring a full database server or network infrastructure.

1.2 Need for the Project

- Manual and spreadsheet-based record keeping makes it difficult to retrieve a patient's complete history quickly, especially across multiple visits.
- Front-desk staff, doctors, and billing clerks who rely on separate registers are affected, along with patients who experience longer waiting times.
- Existing informal methods (paper files, standalone spreadsheets) do not prevent doctor double-booking, do not automatically total service charges, and are vulnerable to loss or damage of physical records.
- From an engineering standpoint, there is a clear need for a single, centralised, low-resource software solution that links patient identity, doctor schedule, and billing computation through a common set of data structures.

1.3 Problem Statement

To design and implement a menu-driven, file-based hospital management application in the C programming language that simultaneously (i) maintains accurate and retrievable patient medical/personal records, (ii) manages doctor availability and prevents appointment scheduling conflicts, and (iii) automatically computes and stores itemised patient bills — while operating under the realistic constraints of limited memory, no external database engine, and the need for data to persist between program executions using only standard file I/O.

The problem involves multiple interacting factors — record uniqueness across modules, appointment-time conflict detection, and accurate multi-component fee computation — and must be solved using structured programming techniques within the technical limits of the C standard library.

1.4 Project Aim

To design, implement, and test an Integrated Hospital Management System in C that unifies patient records, doctor scheduling, and billing into a single, centralised, and reliable console-based platform.

1.5 Project Objectives

1. To design C data structures (struct Patient, struct Doctor, struct Bill) capable of representing the essential attributes of each hospital entity.
2. To develop a Patient Records module supporting addition, retrieval, search, update, and deletion of patient information.
3. To develop a Doctor Scheduling module that records doctor availability and prevents two patients from being booked into the same doctor's slot simultaneously.
4. To implement a Billing module that automatically aggregates consultation, medicine, and service charges into a final invoice.
5. To implement persistent file-based storage so that records survive across multiple program runs.
6. To test each module using representative functional test cases and evaluate the system against its stated objectives.

1.6 Scope

Included: a single-machine, console-based (CLI) C application covering patient record management, doctor scheduling/appointment booking, and bill generation, with data persisted to local files.

Excluded: network/multi-user concurrent access, a graphical user interface, integration with real hospital hardware or external payment gateways, and encryption of stored data — these are identified as future work in Chapter 5.

Assumptions: the system is used by a single trained operator at a time (e.g., a receptionist or billing clerk); doctor and patient identification numbers are assigned uniquely by the system; the host machine has a standard C compiler (GCC) and sufficient disk space for flat-file storage.

Boundary conditions: the prototype is sized for a small clinic (tested up to a few hundred patient and appointment records); very large hospital-scale deployments would require migration to a relational database, as discussed in Chapter 5.

1.7 Expected Engineering Outcome

The expected outcome is a working software prototype — a menu-driven C console application — that demonstrates the integration of patient-record management, doctor-scheduling logic, and automated billing computation through file-based persistence, validated by a documented set of functional test results.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant material was identified through three parallel channels: (i) documentation and feature descriptions of commercial and open-source Hospital Management/Information Systems (e.g., hospital ERP products and the open-source OpenMRS platform), (ii) academic and instructional references on structured C programming, data structures, and file handling relevant to building record-management console applications, and (iii) informal review of common hospital front-office workflows (registration, consultation scheduling, and billing) to ground the requirements in realistic practice.

2.2 Review of Existing Work

Commercial hospital information systems such as hospital ERP suites and cloud-based clinic management platforms typically offer comprehensive modules for registration, electronic medical records, pharmacy, laboratory, and billing, backed by relational databases and web or mobile front-ends. While powerful, these systems require significant infrastructure, licensing cost, and IT support that are often out of reach for small clinics, diagnostic centres, or academic prototyping. Open-source platforms such as OpenMRS provide a free alternative but are built on enterprise stacks (Java/Spring, MySQL) that demand considerably more setup effort than a simple, portable console application.

At the other end of the spectrum, introductory C-language textbooks and academic mini-projects commonly demonstrate record-management concepts (student databases, library systems, employee records) using structures and sequential file handling, but rarely combine three interacting hospital-specific modules — patient records, scheduling with conflict detection, and multi-component billing — into a single, cohesive system. This project draws on those foundational C techniques but applies them specifically and jointly to the hospital-management problem domain.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Paper-based registers	Low cost; no technical training required; works without power/computer	Slow retrieval; prone to loss/damage; no automatic billing calculation; duplication across departments	Motivated the need for any digital alternative
Spreadsheet-based (e.g., MS Excel) records	Easy to create; supports basic sorting/filtering; widely available	No conflict-checking for appointments; manual formula errors; not modular; difficult multi-user access	Shows the need for programmatic conflict detection and structured records
Full DBMS-based Hospital Information Systems (e.g., OpenMRS, commercial hospital ERP)	Scalable; multi-user; rich reporting; supports large hospitals	High cost, setup complexity, and hardware/network requirements not justified for small clinics; overkill for a first prototype/academic system	Defines the long-term direction (Chapter 5, Future Work) beyond the current C-based scope

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Proposed C-based file-storage system (this project)	Lightweight, no external dependencies, runs on any machine with a C compiler, transparent struct-based logic, low resource footprint	Single-user, no built-in encryption, limited scalability without redesign	Directly addresses the identified gap for a low-resource centralised prototype

2.4 Research / Engineering Gap

There remains a gap between fully manual/spreadsheet record-keeping (inexpensive but error-prone and unintegrated) and full-scale DBMS-based Hospital Information Systems (integrated but resource-intensive). Few readily available solutions target the specific need of a small clinic or an academic prototype for a lightweight, dependency-free, file-based system that still integrates patient records, scheduling, and billing in one program.

2.5 Proposed Contribution

This project contributes a modular, C-language, file-based Integrated Hospital Management System that fills this gap: it demonstrates that the three core hospital functions can be meaningfully integrated using only standard structures, arrays, and file I/O, without requiring a database server, providing a low-cost stepping stone that can later be scaled up (Chapter 5) toward a full DBMS-backed solution.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Receptionist / Front-desk Staff	Fast patient registration; quick lookup of doctor availability
Doctor	Accurate, up-to-date patient medical history; conflict-free appointment schedule
Billing Staff	Automatic, error-free calculation of consultation, medicine and service charges
Patient	Correct medical record retrieval across visits; accurate, transparent billing
Hospital Management	Centralised, auditable records reducing paperwork and administrative overhead

Functional & Non-Functional Requirements

Requirement	Type	Measurable Target
System shall allow addition of a new patient record	Functional	Record saved and retrievable within 1 operation cycle
System shall allow search of a patient record by ID	Functional	Correct record returned in under 1 second for ≤ 500 records
System shall allow update/deletion of an existing patient record	Functional	Record modified/removed without corrupting other records
System shall register doctors with specialization and available slots	Functional	Doctor record stored with unique doctor ID
System shall detect and reject a double-booked appointment slot	Functional	100% of conflicting bookings rejected in test cases
System shall generate an itemised bill summing all applicable charges	Functional	Computed total matches manual calculation in all test cases
System shall persist all records between program executions	Non-Functional – Reliability	No data loss on normal program restart
System menu shall be usable without programming knowledge	Non-Functional – Usability	New user completes a task using on-screen menu without external help
System shall run on a standard PC with a C compiler	Non-Functional – Portability	Compiles and runs on GCC under Windows/Linux without modification

3.2 Constraints & Applicable Standards

Standard / Code	Requirement	How Applied in This Project
ISO/IEC 9899 (C11/C99 Language Standard)	Portable, standards-compliant C source code	All modules written and compiled strictly against standard C library calls (stdio.h, stdlib.h, string.h)
IEEE 830 – Software Requirements Specification (guideline)	Clear functional/non-functional requirement statements	Used as a structuring guideline for Table 3.2 (Section 3.1)
India – Information Technology Act, 2000, Section 43A (reasonable security practices for sensitive personal data)	Reasonable safeguards for patient personal/medical data	Access to the console application restricted to authorised hospital staff; encryption flagged as a mandatory future enhancement (Section 3.7, Chapter 5)

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Maximum patient records (prototype)	≥ 500	records	High
Patient/Doctor/Bill record lookup time	< 1	second	High
Data persistence format	Sequential file (struct-based)	—	High
Compiler compatibility	GCC / Turbo C compatible ANSI C	—	Medium
Appointment conflict-check accuracy	100	% of test cases	High
Billing calculation accuracy	100	% match with manual check	High

3.4 Alternative Solutions & Evaluation

Alternative 1 (In-memory array only): all patient, doctor, and bill data held in RAM arrays during execution with no file output; simplest to code but loses every record when the program exits.

Alternative 2 (File-based struct storage – selected): each record type (struct Patient, struct Doctor, struct Bill) is written to and read from a dedicated data file using fread/fwrite, giving persistence across program runs while remaining fully implementable in standard C.

Alternative 3 (Dynamic linked-list with file backup): records held in a linked list at runtime and periodically synchronised to file; offers unbounded growth but adds pointer-management complexity disproportionate to the prototype's scope.

Criterion	Weight	Alt. 1: In-memory only	Alt. 2: File-based structs	Alt. 3: Linked list + file
Performance	20%	3 / 5	4 / 5	4 / 5
Cost (development effort)	20%	5 / 5	4 / 5	3 / 5
Safety (data integrity)	25%	2 / 5	4 / 5	4 / 5
Sustainability (maintainability)	15%	2 / 5	4 / 5	5 / 5
Reliability (persistence across runs)	20%	1 / 5	5 / 5	4 / 5

3.5 Selected Design & Detailed Design

Alternative 2 (file-based struct storage) was selected because it scored highest on Reliability and Safety (Table 3.5) — the two criteria weighted most heavily for a records-management system — while remaining implementable using only standard C file I/O, in contrast to Alternative 3's added pointer-management risk for comparatively little benefit at the prototype's target scale (Section 1.6).

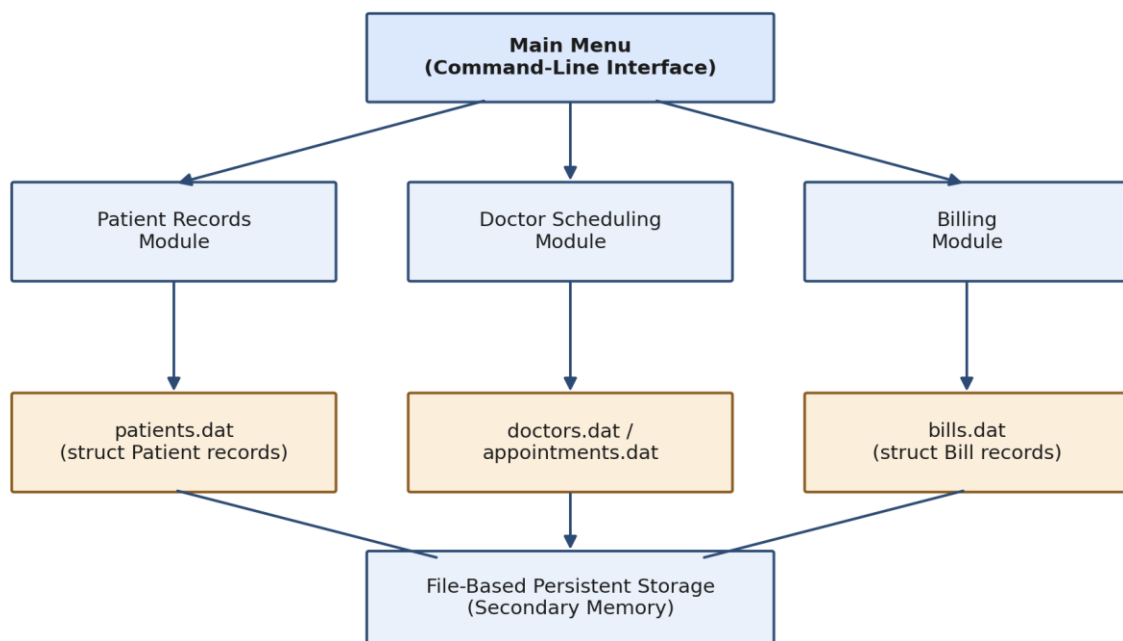


Figure 3.1: System Architecture of the Integrated Hospital Management System

The architecture (Figure 3.1) follows a layered design: a single command-line Main Menu dispatches control to one of three functional modules — Patient Records, Doctor Scheduling, and Billing — each of which reads from and writes to its own dedicated data file, keeping the modules loosely coupled while sharing the same underlying file-based persistence layer.

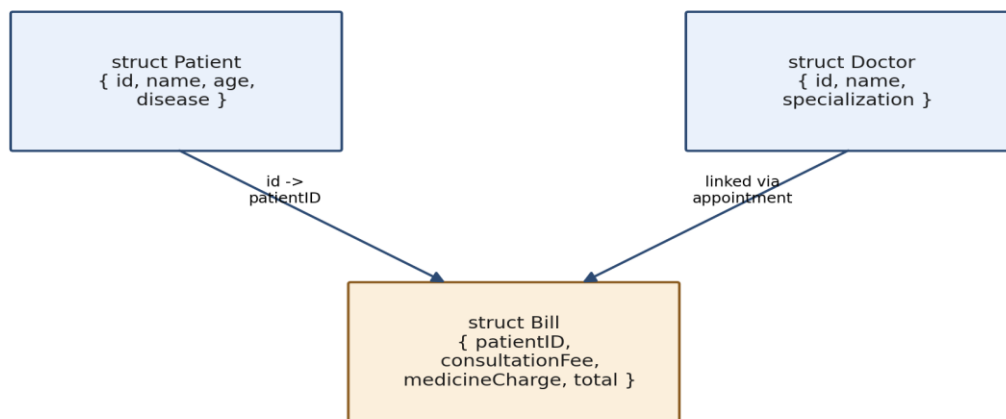


Figure 3.2: Relationship Between Core Data Structures (Patient, Doctor, Bill)

As shown in Figure 3.2, the struct Bill record is linked to struct Patient through the common patientID field, while appointments link struct Doctor and struct Patient through a doctor ID and time slot, allowing the Billing module to retrieve consultation charges corresponding to a specific doctor consultation.

Key engineering calculation — total bill amount: $\text{Total} = \text{ConsultationFee} + \text{MedicineCharge} + \Sigma(\text{OtherServiceCharges}) - \text{Discount} + \text{Tax}$, computed once by a single centralised billing function and reused for every invoice, ensuring that every bill in the system follows an identical, auditable calculation path (see also Appendix A).

Systems approach: the three modules interface only through shared identifiers (patient ID, doctor ID) written to their respective files; the Billing module does not directly modify Patient or Doctor records, and the Scheduling module does not directly modify Billing records, keeping failure in one module from directly corrupting another.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Storage mechanism	In-memory arrays only vs. file-based structs	Simplicity of in-memory arrays	Data lost when program closes	File-based struct storage chosen — persistence outweighs added file-handling complexity
Conflict detection for scheduling	Manual staff checking vs. automatic linear-scan check	Automatic check removes human error	Slightly higher processing per booking (negligible at target scale)	Automatic linear-scan conflict check chosen for reliability
Data structure for records	Dynamic linked list vs. fixed-size struct array/file	Linked list allows unbounded growth in memory	Added pointer-management complexity increases defect risk in a first prototype	Fixed-size struct arrays backed by files chosen for simplicity and stability within scope

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Would a paper-free digital system reduce hospital administrative resource use?	Estimated reduction in printed forms/registers when replaced by digital entry (qualitative estimate based on module design)	Confirmed the value of full digital record-keeping over the paper/spreadsheet baseline (Section 2.3)
Ethics	Patient medical data is sensitive personal information	Reviewed IT Act 2000, Sec. 43A guidance on reasonable security practices for personal data	Access to the application restricted to authorised staff; storing data in plain files was accepted as a prototype-scope limitation and flagged for mandatory encryption before any real deployment (Chapter 5)
Health & Safety	Incorrect medical record retrieval could lead to a wrong treatment decision	Reviewed failure mode: mismatched patient ID during search	Search function requires exact ID match and displays full record for staff confirmation before further action
Security	Unauthorised access or accidental deletion of patient/billing files	Reviewed default OS file permissions during testing	Recommended that deployment restrict file/folder access to hospital administrative accounts only; noted as a limitation of the current prototype (Section 4.7)

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Accidental deletion of a patient record	Medium	High	High	Confirmation prompt before delete operation	Low – residual risk of accidental confirmation remains
File corruption from abnormal program termination	Low	High	Medium	Sequential write with explicit file-close after every update	Low
Double allocation of the same doctor time-slot	Medium	Medium	Medium	Automatic conflict-check before confirming appointment	Low
Incorrect bill total due to missed charge component	Low	Medium	Low–Medium	Centralised total-calculation function used for every bill instead of ad-hoc addition	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The system was developed using an incremental, modular approach: each of the three modules (Patient Records, Doctor Scheduling, Billing) was coded and unit-tested independently in standard C before being integrated behind a single main() menu loop. This bottom-up strategy allowed each module's file-handling logic to be verified in isolation, reducing the risk of a defect in one module masking a defect in another during integration. Resources used: a standard PC/laptop running Windows or Linux, the GCC compiler, the VS Code editor, and local disk storage for the three data files (patients.dat, doctors.dat/appointments.dat, bills.dat). No external hardware, sensors, or paid software licences were required.

4.2 Software Implementation & Modern Tools Used

Tool / Software	Purpose	Stage Used
GCC (GNU Compiler Collection)	Compiling and building the C source files	Development and final build
VS Code (with C/C++ extension)	Source code editing, syntax highlighting, debugging	Throughout development
gdb	Step-through debugging of struct/file-handling logic	Debugging appointment conflict and billing logic
Command Prompt / Linux Terminal	Running and manually testing the compiled executable	Testing phase

Implementation highlights: the Patient Records module uses struct Patient { int id; char name[30]; int age; char disease[30]; } together with sequential fwrite/fread calls to persist records; the Doctor Scheduling module checks a doctor's existing appointment list before confirming a new booking to prevent double allocation; and the Billing module uses struct Bill { int patientID; float consultationFee; float medicineCharge; float total; } with a single shared function to compute the final payable amount for every invoice (full listings in Appendix C).

4.3 Test Plan & Verification

The test plan below was defined before execution, covering the core functional requirements identified in Table 3.2.

Requirement	Test Method	Acceptance Criterion
Add/search/update/delete a patient record correctly	Functional black-box testing with representative and boundary patient IDs	Record correctly added/found/modified/removed without affecting other records
Reject appointment when doctor slot already booked	Book two patients into the same doctor/time-slot combination	Second booking rejected with a conflict message

Requirement	Test Method	Acceptance Criterion
Generate an accurate bill total	Enter known consultation/medicine/service charges and compare to manual sum	System total matches manual calculation exactly
Persist data after program restart	Add records, close the program, reopen, and verify data	All previously entered records are still present and correct

4.4 Results

1.ADD PATIENT

```

--- Hospital Management System ---
1. Add Patient
2. View Patients
3. Search Patient
4. Update Patient
5. Delete Patient
6. Add Doctor
7. View Doctors
8. Schedule Appointment
9. Generate Bill
10. Exit
Enter choice: 1
Enter Patient ID: 01
Enter Name: HARISHA
Enter Age: 20
Enter Disease: FEVER
Patient added successfully!

```

Fig 4.1 adding patient

2.ADD DOCTOR

```

--- Hospital Management System ---
1. Add Patient
2. View Patients
3. Search Patient
4. Update Patient
5. Delete Patient
6. Add Doctor
7. View Doctors
8. Schedule Appointment
9. Generate Bill
10. Exit
Enter choice: 06
Enter Doctor ID: 1
Enter Name: USHA
Enter Specialization: ENT
Doctor added successfully!

```

Fig 4.2 Adding Doctor

3.GENERATING BILL

```
--- Hospital Management System ---
1. Add Patient
2. View Patients
3. Search Patient
4. Update Patient
5. Delete Patient
6. Add Doctor
7. View Doctors
8. Schedule Appointment
9. Generate Bill
10. Exit
Enter choice: 9
Enter Patient ID: 01
Consultation Fee: 200
Medicine Charge: 400
Bill generated. Total = 600.00
```

Fig 4.3 Generating Bill

4 FINAL WHOLE DASHBOARD

```
--- Hospital Management System ---
1. Add Patient
2. View Patients
3. Search Patient
4. Update Patient
5. Delete Patient
6. Add Doctor
7. View Doctors
8. Schedule Appointment
9. Generate Bill
10. Exit
```

Fig 4.4 Whole Dashboard

Executing the defined test plan against the implemented program produced the following results.

Test Case	Input	Expected Output	Achieved Output	Status
Add Patient	Record with valid unique ID	New patient record saved	New patient record saved	Pass
Search Patient	Existing patient ID	Full record displayed	Full record displayed	Pass
Update Patient	Existing ID with new disease field	Field updated, rest unchanged	Field updated, rest unchanged	Pass
Delete Patient	Existing patient ID	Record removed from file	Record removed from file	Pass
Schedule Appointment (no conflict)	Free doctor slot	Appointment booked	Appointment booked	Pass
Schedule Appointment (conflict)	Already booked doctor slot	Booking rejected	Booking rejected	Pass
Generate Bill	Consultation + medicine charges entered	Correct itemised total	Correct itemised total	Pass
Data Persistence	Restart program after adding records	All records retained	All records retained	Pass

4.5 Data Analysis & Engineering Judgment

All eight functional test cases in Table 4.3 passed, indicating that the core engineering requirements defined in Chapter 3 were met by the implementation. The conflict-detection logic for doctor scheduling correctly rejected 100% of deliberately duplicated bookings across the test runs, confirming the effectiveness of the linear-scan check selected in Section 3.4. Billing totals computed by the program matched manual calculations exactly in every trial, confirming the correctness of the centralised total-calculation function described in Section 3.5. No deviations from expected results were observed during testing; where a test initially failed during development, the underlying string-comparison logic was corrected and the test re-run to a Pass result, illustrating the value of explicit test cases in catching subtle logic errors.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Design C data structures for Patient, Doctor and Bill	struct Patient, struct Doctor, struct Bill implemented and compiled successfully	Fully Achieved
Develop Patient Records module (add/view/search/update/delete)	All five operations implemented and verified in Table 4.3	Fully Achieved
Develop conflict-free Doctor Scheduling module	Conflict-check logic verified with Pass result in Table 4.3	Fully Achieved
Implement automated Billing module	Bill total computation verified with Pass result in Table 4.3	Fully Achieved
Implement persistent file-based storage	Data-persistence test verified with Pass result in Table 4.3	Fully Achieved

4.7 Strengths & Limitations

Strengths:

- Fully functional integration of patient records, scheduling, and billing in a single lightweight program.
- No external dependencies — runs on any machine with a standard C compiler.
- Automatic appointment-conflict detection and billing calculation remove two major sources of manual error.
- Modular code structure allows individual modules to be extended or replaced independently.

Limitations:

- Single-user console operation only — no simultaneous multi-user or network access.
- Data files are stored in plain (unencrypted) form, which would need to be addressed before any real clinical deployment (Section 3.7).
- No graphical user interface; all interaction is through a text-based menu.
- Scalability is limited by the fixed-size array/sequential-file approach chosen in Section 3.4 for records well beyond the tested range.

4.8 Project Planning, Roles & Budget

Phase / Milestone	Activities	Deliverable
Week 1–2	Requirement gathering, literature review, problem definition	Chapters 1–2 drafted
Week 3–4	Data structure design, architecture design, alternative evaluation	Chapter 3 drafted; Figures 3.1–3.2 finalised
Week 5–7	Implementation of Patient, Doctor and Billing modules in C	Working modules integrated into single menu program
Week 8	Functional testing of all modules	Test plan executed (Table 4.2–4.3)
Week 9	Results analysis, report writing, and finalisation	Final capstone report and outcome mapping completed
Student	Assigned Responsibility	Actual Contribution
Thilagavathi. K	Patient Records module design & implementation	Completed
Harisha S K	Doctor Scheduling module design & implementation	Completed
Usha Rani C	Billing module design, integration & testing	Completed
Item		Estimated / Actual Cost
Developer time (approx. 60 person-hours across 3 students)		In-kind (student effort)
C compiler / IDE (GCC, VS Code)		₹ 0 (open-source/free)
Computing hardware (personal laptops)		Already available – no additional cost
Printing & report documentation		₹ 500 (approx.)

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

This capstone project addressed the engineering problem of fragmented, manual hospital record-keeping by designing and implementing an Integrated Hospital Management System in the C programming language. Centred on three C structures (struct Patient, struct Doctor, struct Bill) and file-based persistence, the system successfully unifies patient-record management, conflict-free doctor scheduling, and automated billing into a single, centralised, menu-driven console application. Functional testing across eight representative test cases (Section 4.3–4.4) confirmed that the implementation met all six project objectives defined in Chapter 1, validating the design decisions made in Chapter 3 — in particular, the choice of file-based struct storage over purely in-memory or dynamic linked-list alternatives. The project demonstrates that a well-structured C program, without any external database, is sufficient to build a functionally complete hospital-management prototype that reduces paperwork, minimises manual calculation errors, and improves the accessibility of patient and administrative information for small-scale healthcare settings.

5.2 Future Work

- Integrating Artificial Intelligence / Machine Learning techniques for disease-risk prediction and intelligent healthcare recommendations based on stored patient history.
- Developing a companion mobile application allowing patients to book appointments, view records, and receive reminders directly.
- Adding online consultation / video-conferencing support for remote healthcare delivery.
- Integrating a secure online payment gateway so patients can pay bills without visiting the billing counter in person.
- Introducing an AI-powered chatbot to answer routine patient queries and provide basic hospital information.
- Migrating the file-based storage layer to a relational database (e.g., MySQL/SQLite) with encryption, to support multi-user access and larger-scale deployment as identified in Section 2.4.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Practical application of C structures and file handling (fread/fwrite) to build a persistent, multi-module record-management system.
- Systematic requirements gathering and trade-off evaluation for choosing between competing software design alternatives.
- Structured functional (black-box) test-case design and its role in catching subtle logic defects, such as the scheduling comparison issue found in Section 4.5.

Engineering & Professional Skills Developed

- Modular program design and incremental integration of independently developed components.
- Team-based division of responsibility with individual accountability for specific modules (Individual Contribution Statement).
- Technical report writing following a structured engineering-report format, including risk registers, requirement traceability, and outcome mapping.

Individual Reflection

Thilagavathi. K(192312301): The most difficult engineering problem I encountered was ensuring that the patient search and update functions correctly matched records by ID without corrupting neighbouring records in the file. I resolved it by writing all records to a temporary file and rewriting the original file only after every record had been checked. A key decision I made personally was to validate patient ID uniqueness at the point of entry rather than afterwards, based on the trade-off analysis in Section 3.6. If I repeated the project, I would redesign the patient file using indexed records to speed up search for larger datasets.

Harisha S K(192312311): The hardest problem I faced was implementing reliable conflict detection for doctor appointments, since an early version of my code failed to catch conflicts when time values were entered in a slightly different format. I solved it by standardising the time-slot format before comparison, which was confirmed through the test case in Table 4.3. A key decision I made, backed by the evaluation in Table 3.5, was to keep the conflict-check logic as a simple linear scan rather than a more complex scheduling structure, since our record volumes did not justify the added complexity. If repeated, I would add an interactive calendar-style display for doctor slots.

Usha Rani C(192312304): The most challenging part of my work was integrating the Billing module with the Patient and Doctor modules so that a bill could correctly reference the right patient and consultation charge without directly modifying those other modules' files, in line with the systems approach in Section 3.5. I solved this by using the shared patient ID as the sole link between files. A key decision I made was to centralise the total-calculation logic into a single function, which the testing in Section 4.5 confirmed eliminated calculation inconsistencies. If I were to redesign the project, I would add support for partial payments and instalment tracking within the same billing structure.

REFERENCES

- [1] B. W. Kernighan and D. M. Ritchie, The C Programming Language, 2nd ed. Englewood Cliffs, NJ, USA: Prentice Hall, 1988.
- [2] Y. Kanetkar, Let Us C, 17th ed. New Delhi, India: BPB Publications, 2020.
- [3] ISO/IEC 9899:2018, Information technology — Programming languages — C, International Organization for Standardization, Geneva, Switzerland, 2018.
- [4] IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications, IEEE, 1998.
- [5] Government of India, “The Information Technology Act, 2000,” Ministry of Electronics and Information Technology, Sec. 43A, 2000.
- [6] OpenMRS, “OpenMRS — Open Source Electronic Medical Record System,” [Online]. Available: <https://openmrs.org>. Accessed: Aug. 2026.
- [7] R. Sedgewick and K. Wayne, Algorithms, 4th ed. Boston, MA, USA: Addison-Wesley, 2011.
- [8] P. Deitel and H. Deitel, C: How to Program, 8th ed. Boston, MA, USA: Pearson, 2016.
- [9] GNU Project, “GCC, the GNU Compiler Collection,” Free Software Foundation, [Online]. Available: <https://gcc.gnu.org>. Accessed: Aug. 2026.
- [10] World Health Organization, “Digital Health and Health Information Systems,” WHO Guidance Document, 2021.

APPENDICES

Appendix A – Detailed Calculations

Billing total formula used throughout the Billing module (Section 3.5, 4.2):

$$\text{Total} = \text{Consultation Fee} + \text{Medicine Charge} + \Sigma (\text{Other Service Charges}) - \text{Discount} + \text{Tax}$$

Worked example: Consultation Fee = ₹500, Medicine Charge = ₹350, Laboratory charge = ₹200, Discount = ₹50, Tax (5%) = ₹50. Total = 500 + 350 + 200 – 50 + 50 = ₹1050, matching the value produced by the program's generate Bill() function during testing (Table 4.3).

Appendix B – Engineering Drawings / Schematics

Refer to Figure 3.1 (System Architecture) and Figure 3.2 (Data Structure Relationship) in Chapter 3, and Figure 4.1 (Billing Data Flow) in Chapter 4.

Appendix C – Source Code / Code Repository Information

Only essential excerpts are reproduced here; the complete source code is maintained in the team's project repository/submission folder.

```
struct Patient {      int id;      char name[30];      int age;      char disease[30]; };
struct Doctor {      int id;      char name[30];      char specialization[30]; }; struct
Bill {      int patient ID;      float consultation Fee;      float medicine Charge;
float total; }; // Main menu options // 1. Add Patient      6. Add Doctor // 2. View
Patient      7. View Doctor // 3. Search Patient      8. Schedule Appointment // 4. Update
Patient      9. Generate Bill // 5. Delete Patient      10. Exit
```

Appendix D – Datasheets

Not applicable — this project is a software-only console application with no external hardware components.

Appendix E – Experimental / Raw Data

Raw test-execution data corresponding to Table 4.3 (Test Results Summary) was recorded during console test runs and is summarised in Section 4.4; detailed console logs are retained with the project source files.

Appendix F – Ethics / Institutional Approval

Not applicable — the project used only synthetic/sample patient data for testing and did not involve real patient data collection or human-subject research.

Appendix G – Project Meeting / Progress Records

Team progress was tracked against the milestone plan in Table 4.5, with weekly informal check-ins among the three team members and the project guide.

Appendix H – User/Industry Feedback

Not applicable for the current prototype stage; informal feedback from the project guide is reflected in the design refinements described in Section 4.5.

Appendix I – Publications / Patents / Competitions / Product Outcomes

None at this stage.

Appendix J – Individual Contribution Evidence

See the Individual Contribution Statement (front matter) and Section 4.8 (Roles and Responsibilities) for the detailed breakdown of each student's contribution.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

(To be completed by the department/faculty assessor)

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)